FASE 2 · PROYECTO 04

ring0

Tu primer módulo del kernel:
entorno, Kbuild, /proc y cómo leer un oops

Versión 1.0 · Roadmap Sistemas Linux

Resumen

Antes de escribir un driver necesitas un laboratorio en el que romper cosas sea gratis. En este proyecto montas una VM de desarrollo, escribes tus primeros módulos (LKM), expones información en `/proc` y provocas a propósito un *kernel oops* para aprender a leerlo. Es corto, pero todo lo de la Fase 2 depende de él.

CAPÍTULO I

Introducción

Un *Loadable Kernel Module* es código objeto que se enlaza en caliente con el kernel en ejecución. Una vez cargado no hay frontera de protección: tu código corre en Ring 0 con el mismo poder que el resto del kernel. No hay libc, ni `malloc`, ni `printf`, ni red de seguridad: un puntero nulo no produce un segfault, produce un oops que puede dejar el sistema inestable o directamente en *kernel panic*.

Las reglas cambian: la memoria se pide con `kmalloc`/`kzalloc` indicando el contexto (`GFP_KERNEL`), se imprime con `pr_info` y similares, la pila es pequeña (16 KiB en x86-64) y nunca puedes confiar en un puntero que venga de user space.

CAPÍTULO II

Instrucciones generales

- **Todo el trabajo de kernel se hace dentro de una VM.** Nunca cargues tus módulos en tu máquina principal.
- Lenguaje C según el estilo del kernel. `scripts/checkpatch.pl --no-tree -f` sobre tus `.c` no debe devolver ningún error ni warning.
- Compila contra las cabeceras del kernel de la VM con un Makefile Kbuild (`obj-m`). La compilación no debe producir ningún warning.
- Licencia: declara `MODULE_LICENSE("GPL")`, además de `MODULE_AUTHOR` y `MODULE_DESCRIPTION`.
- Un módulo debe poder cargarse y descargarse 1000 veces seguidas sin fugas ni errores.

CAPÍTULO III

Ejercicios

Ejercicio	00: lab
Entrega	`ex00/ENV.md`
Permitido	QEMU/KVM, VirtualBox o UTM; `apt`

Monta una VM Debian 12 o Ubuntu 24.04 (2 vCPU, 2–4 GiB de RAM). Instala `build-essential` y `linux-headers-$(uname -r)`, configura acceso por SSH desde tu máquina y crea un **snapshot** limpio. Si la VM usa Secure Boot, desactívalo: los módulos sin firmar no se cargarán.

En `ENV.md` documenta: versión del kernel, cómo restauras el snapshot, cómo copias código a la VM (carpeta compartida, `rsync` o `git`) y tu ciclo de trabajo (editar → compilar → `insmod` → `dmesg` → `rmmod`).

Ejercicio	01: hello
Entrega	`ex01/` · `hello.c`, `Makefile`
Permitido	`module_init`, `module_exit`, `pr_info`, `pr_err`

El clásico, bien hecho. Al cargarse, `hello.ko` imprime en el log del kernel el PID y el nombre del proceso que lo ha cargado (pista: la variable `current`). Al descargarse, se despide.

```
# Recuerda: las líneas de receta empiezan con TAB, no con espacios
obj-m += hello.o
KDIR  ?= /lib/modules/$(shell uname -r)/build

all:
	$(MAKE) -C $(KDIR) M=$(CURDIR) modules
clean:
	$(MAKE) -C $(KDIR) M=$(CURDIR) clean
```

Comprueba el resultado con `modinfo hello.ko`, `lsmod`, `dmesg -w` y `cat /proc/modules`. Explica en el README qué significa la marca de *taint* que aparece al cargarlo (`cat /proc/sys/kernel/tainted`).

Ejercicio	02: params
Entrega	`ex02/`
Permitido	`module_param`, `MODULE_PARM_DESC`, `kstrtoint`

Un módulo con dos parámetros: `count` (entero, 1 a 100, por defecto 3) y `who` (cadena, por defecto «mundo»). Al cargarse imprime `count` veces «hola, <who>».

- Si `count` está fuera de rango, la carga debe **fallar** con `-EINVAL` y un mensaje claro: el módulo no se queda cargado.
- `count` debe ser legible y modificable en `/sys/module/params/parameters/count` por root. Reflexiona: ¿qué ocurre si alguien escribe ahí un valor fuera de rango? Bonus: valídalo con `module_param_cb`.

Ejercicio	03: procstat
Entrega	`ex03/`
Permitido	`proc_create`, `proc_remove`, `struct proc_ops`, `seq_file`, `jiffies`, `atomic_t`

Crea el fichero `/proc/ring0_stats`. Cada lectura muestra:

```
loaded_at_jiffies: 4295123456
seconds_loaded:    42
reads:             7
last_reader:       cat (pid 1834)
```

- Usa la interfaz `seq_file` (`single_open` basta). `reads` se incrementa de forma segura aunque haya lecturas concurrentes.
- Desde el kernel 5.6 las entradas de `/proc` usan `struct proc_ops`, no `struct file_operations`. Si un tutorial usa la segunda, está desactualizado: acostúmbrate a contrastar con elixir.bootlin.com para la versión exacta de tu kernel.
- Al descargar el módulo, el fichero desaparece. Prueba qué ocurre si haces `rmmod` mientras otro terminal mantiene el fichero abierto (`exec 3< /proc/ring0_stats`).

Ejercicio	04: oops
Entrega	`ex04/` · `buggy.c`, `OOPS.md`
Permitido	lo anterior + `gdb`, `addr2line`, `objdump`

En la VM (con snapshot hecho), escribe un módulo que, al leer `/proc/buggy`, desreferencia un puntero nulo dentro de una función auxiliar. Cárgalo, provoca el fallo y captura el oops completo de `dmesg`.

En `OOPS.md`, explica línea a línea: el tipo de fallo, `RIP`, el formato `funcion+0xoffset/0xtamaño`, la *call trace*, los registros, el `CR2` y los flags de *taint*. Después, localiza la línea exacta de código que falló a partir del offset:

```
$ gdb buggy.ko
(gdb) list *(helper_read+0x1a)
```

> **NOTA**
> Tras un oops, el kernel puede seguir funcionando, pero ya no es fiable: el proceso que lo provocó murió con locks quizá tomados. Restaura el snapshot en lugar de seguir trabajando sobre esa VM.

CAPÍTULO IV

Parte bonus

- **Compila tu propio kernel** (rama estable reciente) con `CONFIG_KASAN`, `CONFIG_PROVE_LOCKING`, `CONFIG_DEBUG_KMEMLEAK` y `CONFIG_DEBUG_INFO`. Arráncalo con `virtme-ng` o con `qemu -kernel`. Será tu entorno principal en `ringdev`: KASAN y lockdep encuentran bugs que de otro modo tardarías días en ver.
- Depuración con `gdb` del kernel en vivo: QEMU con `-s -S`, arranque con `nokaslr` y los scripts `lx-symbols` y `lx-dmesg`. Pon un breakpoint en la función `read` de tu módulo y recorre la pila.
- Un módulo con una fuga deliberada de `kmalloc` y su detección con `/sys/kernel/debug/kmemleak`.

CAPÍTULO V

Evaluación

Prueba	Esperado
Demostración del ciclo de trabajo	Restaurar snapshot, compilar, cargar y ver el log en menos de un minuto.
`for i in $(seq 1000); do insmod hello.ko && rmmod hello; done`	Sin errores; memoria estable en `/proc/meminfo`.
`insmod params.ko count=500`	Carga rechazada, mensaje en `dmesg`, módulo no listado en `lsmod`.
Lecturas concurrentes de `/proc/ring0_stats` en bucle desde 8 procesos	Contador coherente, sin warnings.
`checkpatch.pl`	0 errores, 0 warnings.
Explicación de `OOPS.md`	El evaluado localiza la línea del fallo delante del evaluador.
Pregunta: ¿por qué no puedes usar `printf` ni `malloc` en el kernel? ¿Qué pasa si duermes con un spinlock tomado?	Respuesta precisa.